



Configurazione, Contesto e Flusso di Analisi del Progetto SO

Monitoraggio Bancario in Tempo Reale

Sommario Esecutivo

Questa relazione descrive con taglio tecnico e operativo come è strutturato il progetto, da dove provengono i dati, in che modo vengono normalizzati, e come vengono trasformati in decisioni. Il documento non replica i dettagli dei singoli problemi (già presenti nel README), ma chiarisce il contesto, le motivazioni architetturali e il funzionamento complessivo.

L'obiettivo è dare una visione completa della pipeline: ingresso dati, normalizzazione, monitoraggio, gestione del rischio e output finale. L'intero sistema è progettato per funzionare in tempo reale e con logica di escalation cumulativa, favorendo la tempestività rispetto all'analisi storica.

Obiettivi del Sistema

Il sistema è stato progettato per rispondere alle esigenze critiche del monitoraggio bancario in tempo reale. Gli obiettivi principali sono:

- Monitorare un flusso di eventi in tempo reale
- Ridurre al minimo il tempo di rilevamento delle anomalie
- Definire regole semplici ma efficaci
- Accumulare il rischio nel tempo per evidenziare recidività
- Generare output strutturato e leggibile

Contesto Operativo

Il progetto simula un contesto bancario in cui le anomalie devono emergere in pochi secondi e non dopo una lunga elaborazione batch. La tempestività è fondamentale per prevenire frodi e garantire la sicurezza dei clienti.

Contesto Operativo

Il sistema assume un contesto in cui il server applicativo riceve richieste HTTP da client diversi (utente web, ATM, app, script automatizzati). Le richieste sono trattate come eventi con campi standardizzati.

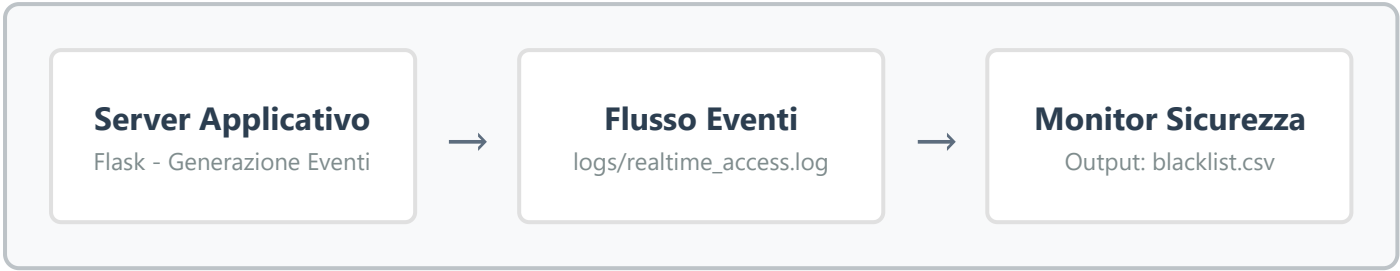
In un ambiente reale, il monitoraggio può basarsi su traffico di rete, log applicativi e database. Qui la scelta è stata deliberatamente semplificata: un solo canale in tempo reale, generato dal server, e letto in streaming dai monitor.

Scelta Architettuale

Questa scelta rende l'osservazione uniforme e consente di isolare la logica di rilevamento da fattori esterni, garantendo maggiore controllo e predicibilità del sistema.

Architettura Logica

L'architettura è composta da tre blocchi principali che operano in modo coordinato ma indipendente:



La separazione è netta: il server produce eventi, i monitor leggono eventi, la blacklist raccoglie decisioni. Ogni blocco ha responsabilità singola, garantendo modularità e manutenibilità.

Struttura del Workspace

Il progetto è organizzato secondo una struttura chiara e logica. I percorsi principali sono:

Componenti Core

- server/server.py** - Server Flask e generazione eventi
- logs/realtime_access.log** - Stream eventi (input unico)
- script/problema_*.sh** - Monitor per ciascuna anomalia

Supporto e Output

- script/run_problem.sh** - Orchestratore monitor + test
- blacklist.csv** - Output cumulativo
- clienti_banca.csv** - Anagrafica (email e 2FA)

Documentazione

Il README descrive i problemi specifici; questa relazione descrive il contesto e la configurazione globale del sistema.

Origine e Formato dei Dati

5.1 Generazione Eventi

Il server riceve richieste HTTP (login, bonifico, prelievo, deposito). Ogni chiamata produce un evento e lo scrive nel file di stream. Il server è responsabile di:

- Acquisire customer_id e parametri dell'operazione
- Leggere l'IP da X-Forwarded-For o remote_addr
- Generare un timestamp coerente
- Scrivere una riga nel log realtime

5.2 Formato del Flusso

Il formato scelto è un record pipe-separated per garantire efficienza e semplicità:

```
timestamp|customer_id|ip|azione|importo|iban|session_duration
```

Motivi della Scelta

- Parsing semplice con strumenti standard (awk, cut)
- Velocità di lettura in streaming
- Compatibilità con script bash

Vantaggi Operativi

- Nessuna dipendenza da librerie esterne
- Facile debugging e ispezione manuale
- Efficienza in ambienti con risorse limitate

5.3 Esempio Concreto

Record di Login

```
2026-02-23T04:01:10|800001|192.168.10.12|LOGIN|||
```

timestamp: 2026-02-23T04:01:10

customer_id: 800001

ip: 192.168.10.12

azione: LOGIN

importo: vuoto (non applicabile)

iban: vuoto (non applicabile)

session_duration: vuoto (non applicabile)

Lettura dello Stream

Ogni monitor utilizza il comando standard per la lettura continua del file:

```
tail -f logs/realtime_access.log
```

Questo comando mantiene una lettura continua del file mentre cresce. Non c'è buffering complesso: il monitor processa la linea appena appare.

6.1 Finestra Temporale

Strategia di Analisi

Per evitare accumuli infiniti, ogni monitor applica una finestra di 60 secondi. Questo consente test rapidi e favorisce analisi a breve termine, mantenendo il sistema reattivo e performante.

6.2 Stato Temporaneo

Quando necessario, i monitor usano file di stato in **logs/** per contare occorrenze. Questi file sono puramente temporanei e non costituiscono input storico stabile.

Gestione del Rischio

7.1 Blacklist

La blacklist è un CSV con header fisso che registra tutti gli eventi rilevanti:

```
timestamp, tipo_elemento, elemento, azione_rilevata, gravita, recidivita, risk_score, stato,
origine
```

7.2 Recidività e Rischio Cumulativo

La logica centralizzata implementa un sistema di accumulo del rischio:

Legge l'ultima recidività per l'elemento

Somma il nuovo rischio al totale esistente

Aggiorna stato a BLOCKED se il totale è ≥ 100

BASSA

Score: 0-30
Monitoraggio

MEDIA

Score: 31-70
Allerta

ALTA

Score: 71-99
Blocco Imminente

CRITICA

Score: ≥ 100
BLOCKED

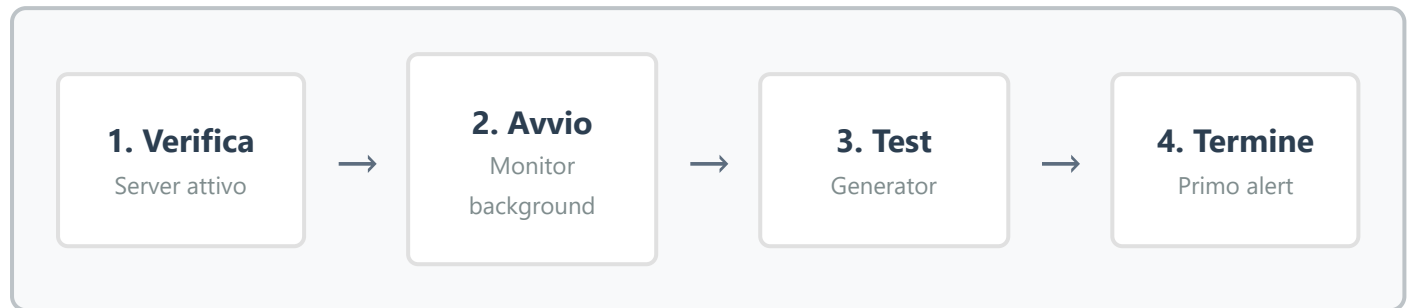
Memoria Operativa

Questo modello introduce una memoria operativa: un segnale medio ripetuto più volte può raggiungere criticità elevata. Anche comportamenti singolarmente non critici diventano rilevanti se ripetuti.

Orchestrazione e Test

8.1 run_problem.sh

Lo script di orchestrazione esegue una sequenza precisa di operazioni:



8.2 Generatori di Test

Ogni generatore è progettato per simulare scenari specifici:

- Usa curl con parametri in query string
- Seleziona account casuali dal CSV
- Invia richieste in modo da superare la soglia

Riproducibilità

I test sono pensati per essere riproducibili e immediati, consentendo validazione rapida delle regole di rilevamento.

Motivazioni Architettureali

9.1 Flusso Unico

Un singolo flusso eventi semplifica la pipeline e garantisce coerenza. Invece di leggere da database o log multipli, si osserva un canale unico, che rappresenta l'azione in tempo reale.

9.2 Regole Deterministiche

Le soglie sono statiche e chiare. Questo rende le decisioni spiegabili e favorisce il debugging e la validazione rapida.

9.3 Escalation Cumulativa

L'accumulo di risk_score crea un ponte tra rilevamento immediato e memoria storica. Anche se il sistema non legge log storici, conserva la recidività nella blacklist.

Vantaggio Chiave

Questa architettura bilancia semplicità operativa e capacità di apprendimento, senza richiedere sistemi complessi di machine learning.

Accenno ai Problemi

I problemi sono documentati in dettaglio nel README. In sintesi, il sistema copre le seguenti categorie di anomalie:

Anomalie Comportamentali

- Schemi AML (Anti-Money Laundering)
- Accessi simultanei da location diverse
- Accessi notturni fuori orario
- Pattern API ripetitivi
- Low & slow attacks

Anomalie Tecniche

- Subnet ATM inattesa
- Brute-force login
- IP pubblici inattesi
- Importo zero (covert channel)
- Incoerenza ATM/bonifico

Questi casi sono stati scelti per coprire comportamenti anomali, coerenza di rete ed attacchi a bassa intensità, rappresentando uno spettro completo di minacce reali.

Comandi Principali e Uso Pratico

Il sistema utilizza comandi standard UNIX/Linux per garantire portabilità e semplicità. Ecco i comandi effettivamente usati nei monitor e test:

- `tail -f` - Lettura continua del flusso eventi
- `awk -F` - Parsing dei campi pipe-separated
- `sort -u` - Estrazione valori unici
- `wc -l` - Conteggio occorrenze
- `grep -q` - Match pattern silenzioso
- `cut -d -f` - Estrazione campi specifici
- `date -d` - Interpretazione timestamp
- `date +%H` - Estrazione ora per analisi temporale
- `curl -s -G --data-urlencode` - Generazione richieste test

Compatibilità

Questi comandi garantiscono compatibilità con un ambiente GNU/Linux standard e riducono dipendenze esterne, rendendo il sistema facilmente deployabile.

Requisiti e Dipendenze

Ambiente di Esecuzione

Sistema Operativo

- Alpine Linux (consigliato)
- Bash shell
- Strumenti GNU standard

Software Applicativo

- Python 3
- Flask (server)
- Pandoc + LaTeX (solo per PDF)

Nota Importante

Il monitoraggio non richiede librerie esterne oltre agli strumenti standard in shell. Questo garantisce massima portabilità e minima superficie di attacco.

Robustezza e Limiti

Il progetto è intenzionalmente semplice per favorire comprensione e manutenibilità. I limiti riconosciuti sono:

Limitazioni Architettureali

- Non usa archiviazione storica come input
- Non calcola trend a lungo periodo
- Non integra feed esterni

Limitazioni Operative

- Non gestisce alert multipli simultanei
- Finestra temporale fissa (60s)
- Nessuna correlazione cross-problema

Contesto e Validità

Questi limiti sono accettabili nel contesto didattico e rendono la pipeline chiara e verificabile. Il sistema dimostra efficacemente i principi di monitoraggio real-time senza la complessità di soluzioni enterprise.

Conclusione

La configurazione proposta offre un contesto chiaro e pratico per studiare anomalie di sicurezza in tempo reale. I dati entrano dal server, vengono normalizzati nello stream, analizzati dai monitor e trasformati in output strutturati con accumulo del rischio.



La relazione mette in evidenza il flusso completo e il ragionamento architetturale, fornendo una base solida per eventuali estensioni future. Il sistema dimostra come principi semplici, applicati con rigore, possano produrre un monitoraggio efficace e manutenibile.

Allegato: Esempio di Riga Blacklist

Di seguito un esempio reale di record nella blacklist, che mostra tutti i campi popolati:

Campo	Valore	Descrizione
timestamp	2026-02-23 04:03:46	Momento del rilevamento
tipo_elemento	ACCOUNT	Tipo di entità monitorata
elemento	800038	ID specifico (customer_id)
azione_rilevata	ACCESSO_NOTTURNO	Tipo di anomalia
gravita	MEDIA	Livello di rischio
recidivita	1	Numero occorrenze
risk_score	30	Punteggio cumulativo
stato	ACTIVE	Stato corrente
origine	PROFILO_RETE	Categoria monitor

Record Completo (CSV)

```
2026-02-23
04:03:46,ACCOUNT,800038,ACCESSO_NOTTURNO,MEDIA,1,30,ACTIVE,PROFILO_RETE,Accesso
anomalo rilevato
```

Sistema di Monitoraggio Bancario Real-Time

Documento tecnico generato per il progetto SO - Configurazione e analisi del flusso di rilevamento anomalie

Pipeline: Ingresso → Normalizzazione → Monitoraggio → Gestione Rischio → Output Strutturato